

综合题

信号量编程

题型1:单进程双信号，两次检查

36、某展览会任何时刻最多可容纳500名参观者，当展览厅中少于500名参观者时，则厅外的参观者可立即进入，否则需在外面等待。参观者进入展览厅时，都必须在入口处登记(并领取资料和礼品)，假定入口处有5位工作人员，每位工作人员每次只能接待一个参观者登记，请用记录型信号量机制实现参观者进程的同步算法。register() 是完成登记并领取资料和礼品的函数; visit() 是完成参观展览的函数; leave() 是表示参观完毕离开的函数。下面已经给出了部分代码，请填写1~2 空白处的代码。注:每一空可能不止一行代码。 <pre>struct semaphore cap, officer; //分别表示展览会容量、工作人员的信号量 (1) void processVisitor() //参观者 { (2) }</pre>	<pre>(1) cap.value = 500; officer.value = 5; (2) wait(cap); // 检查容量 wait(officer); // 检查登记 register(); signal(officer); // 释放登记资源 visit(); leave(); signal(cap); // 释放容量资源</pre>
---	--

题型2:双进程，生产者消费者问题变型，考核数组指针

36.设有无穷多个整数缓冲区（即为无界缓冲池），A进程从输入设备逐个地读入整数并写入缓冲区，B进程则逐个地从缓冲区取出整数进行打印。其中存放整数的变量为item，缓冲区名为buffer，读取过程使用函数getAItem(int* item)来完成，而打印整数使用函数printAItem(int item)来完成。请用记录型信号量机制实现上述两个进程的同步算法。要求：补充完整下列算法程序中带标号处空缺的内容。 （注：每个空缺部分的代码可能是多行代码）。	<pre>struct semaphore full; int buffer[]; // 缓冲区 int in, out; // 缓冲区的入口指针量和出口指针量 (1) void processA() { int item; // 存放整数的变量 (2) } void processB() { int item; // 存放整数的变量 (3) }</pre>	<pre>(1) in = 0; out = 0; full.value = 0; (2) while(TRUE){ getAItem(&item); buffer[in++] = item; signal(full); } (3) while(TRUE){ wait(full); item = buffer[out++]; printAItem(item); }</pre>
---	---	---

题型3:单进程双信号量，初始值1或n，3次检查

假定一个阅览室最多可同时容纳100个人阅读，读者进入和离开阅览室时，都必须在阅览室门口的一个登记表上登记。假定每次只允许一个人登记和注销登记（要求互斥）。用 wait 和 signal 操作编写读者进程的同步算法，补充完成程序

<pre>begin ① _____: semaphore /定义各个信号量/ ② _____ /各个信号量赋初值/ cobegin process Readeri (i=1, 2, ..., n) begin 到达阅览室入口处; ③ _____ ④ _____ 在入口处进行登记; ⑤ _____ 进入阅览室, 进行阅读; 阅读结束, 到达阅览室出口处; ⑥ _____ 在出口处进行注销登记; ⑦ _____ 离开阅览室; ⑧ _____ end; coend; end;</pre>	<pre>(1) x, mutex; semaphore (2) x.value=100 mutex.value=1 到达阅览室入口 (3) wait(x) (4) wait(mutex) 在入口处登记 (5) signal(mutex) 进入阅览室, 进行阅读 阅读结束, 到达阅览室出口 (6) wait(mutex) 在出口处注销登记 (7) signal(mutex) (8) signal(x)</pre>	① 定义信号量x和mutex， x表示阅览室的剩余空位， mutex表示登记表互斥操作 ② x, mutex信号量初始化为 100, 1 ③ 检查阅览室是否有空位可以 进入 ④ 检查登记表是否能操作 ⑤ 登记完解锁，让下一位可以 操作登记表 ⑥ 检查登记表是否能操作 ⑦ 登记完解锁，让下一位可以 操作登记表 ⑧ 完成阅读离开，让下一位读 者进入阅览室
--	--	--

第二章 进程管理

2.1 进程的描述

程序的并发执行

程序的顺序执行

含义：先进入内存的程序先执行，在一个程序执行完毕之前，不能执行其他程序。

特点：顺序性、封闭性、可再现性

程序的并发执行

含义：指在同一时间间隔内运行多个程序。一个程序执行结束前，可以运行其他程序。

宏观并行：用户看到多个程序同时向前推进

微观串行：任意时刻一个CPU上只有一个程序在执行

特点：中断性、失去封闭性、不可再现性。

① 中断性：程序在CPU上执行时，是时断时续的；

② 失去封闭性：系统的状态不再只对正在执行的程序可见；

③ 不可再现性：同一个程序在输入相同的情况下多次运行，可能出现不同的结果。

进程的概念

进程的定义 1) 允许并发执行的程序在某个数据集合上的运行过程 2) 进程=正文段+用户数据段+进程控制块 (PCB)	程序的定义 具有独立功能的一组指令的集合
--	------------------------------------

进程的特征：并发性，动态性，独立性，异步性，结构特性

并发性：多个进程实体能在一段时间间隔内同时运行。

动态性：进程是实体的执行过程

独立性：独立运行和资源调度的基本单位

异步性：进程的**执行时断时续，何时执行、何时暂停都无法预知**

结构特性：进程实体包括用户**正文段、用户数据段和进程控制块**

并发性，是进程和现代操作系统的重要特征

进程和程序的比较

区别：

程序：静态、永久、指令的集合。

进程：动态、暂时、包括正文段、用户数据段、进程控制块。

联系：

进程是程序的一次执行，

一个程序可以对应多个进程；

同一个进程能顺序的执行几个程序。

进程控制块

进程实体包括用户正文段、用户数据段和**进程控制块**

进程控制块

定义：进程控制块是**进程实体的一部分**，是操作系统中**最重要的数据结构**。

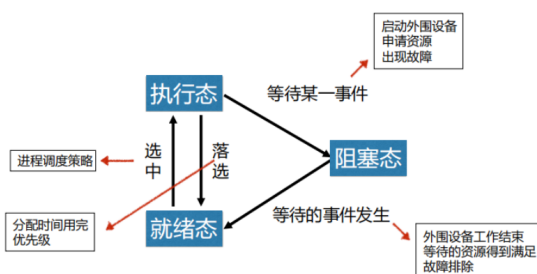
进程控制块中**记录**了操作系统所需要的、用于**描述进程及控制进程运行所需的全部信息**

- 1) 进程标识符信息：用于唯一标识一个进程。
- 2) 处理机状态信息：通用寄存器、指令计数器、程序状态字PSW 和用户栈指针。
- 3) 进程调度信息：包括进程状态信息、进程优先级和进程调度所需的其他信息。
- 4) 进程控制信息：包括程序和数据的地址、进程同步和通信机制、资源清单，以及链接指针。

进程的状态

进程有三种状态：就绪态、执行态、阻塞态；

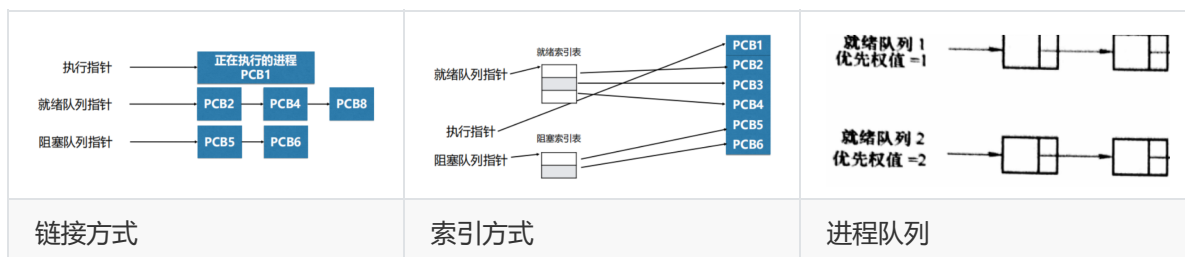
状态转换中，与“阻塞态”相关的都是单向转换的，只能执行态到阻塞态，阻塞态到就绪态。



进程的组织

进程的组织 → 进程控制块的组织

- 1) 链接方式：把系统中具有**相同状态**的进程控制块 PCB 用其中的**链接字**连成一个队列
- 2) 索引方式：系统根据所有进程的状态，建立**索引表**，索引表的每一个**表项**指向一个**PCB物理块**
- 3) 进程队列：把具有相同状态的进程控制块用**队列**组织起来。



2.2 进程的控制

进程的控制

1) 进程的创建

条件：用户登录、作业调度、提供服务、应用请求

- 步骤：① 申请空白 PCB；
② 为新进程分配资源；
③ 初始化进程控制块；
④ 将新进程插入就绪队列。

当新进程被创建时，有两种执行可能：

- ① 父进程与子进程并发执行。
② 父进程等待，直到某个或全部子进程执行完毕

有两种地址空间：

- ① 子进程共享父进程的地址空间。
② 子进程拥有独立地址空间

2) 进程的阻塞

条件：请求系统服务、启动某种操作、新数据尚未到达、无新工作可做。

- 步骤：① 将进程的状态改为阻塞态；
② 将进程插入相应的阻塞队列；
③ 转到进程调度程序，从就绪进程中选择进程为其分配 CPU。

3) 进程的唤醒

- 步骤：① 将进程从阻塞队列中移出；
② 将进程状态由阻塞态改为就绪态；
③ 将进程插入就绪队列。

4) 进程的终止

- 条件：① 进程正常执行完毕，调用终止进程的系统调用，请求操作系统删除该进程；
② 一个进程调用适当的系统调用，终止另外一个进程。

- 步骤：① 从进程PCB 中读进程状态；
② 若进程正在执行，则终止进程执行；
③ 释放资源。
④ 将终止进程的PCB移出。

2.3 操作系统内核

操作系统内核

操作系统内核是计算机硬件的第一次扩充，内核执行操作系统与硬件关系密切，执行频率高的模块，常驻**内存**。

1) 支撑功能

中断处理、时钟管理 和 原语操作。

2) 资源管理功能

进程管理、存储器管理 和 设备管理。

中断

中断是什么

中断是**改变处理器执行指令顺序**的一种事件。

出现中断时，计算机**停止现在程序的运行**，转向**对这些中断事件的处理**，处理结束后再**返回到现行程序的间断处**。

中断的作用

引入中断机制后，CPU可以与其他设备并行工作，能有效**提高CPU的利用率**，改善系统性能，**支持系统的异步**操作。

中断分类

同步中断（也称内部中断或异常）

异步中断（也称外部中断）

- ① 外部可屏蔽中断，
- ② 外部不可屏蔽中断

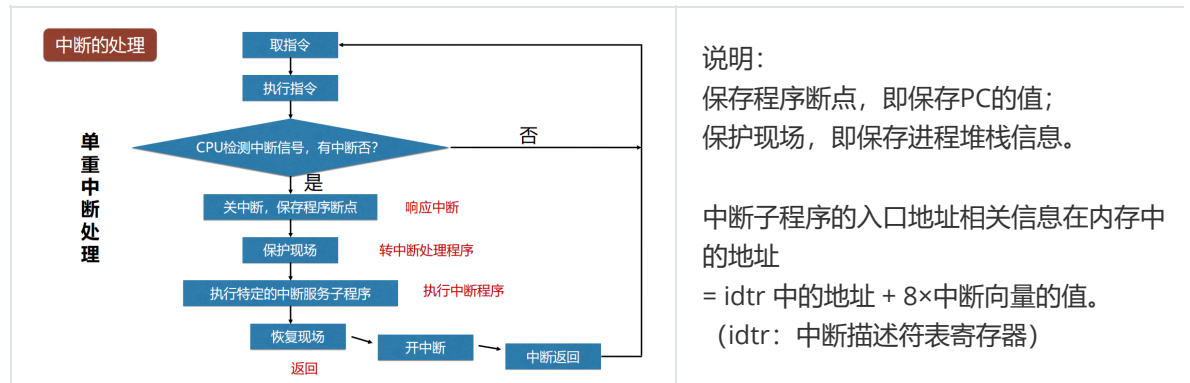
引起中断原因

1) 人为设置中断，2) 程序性事故，3) 硬件故障，4) I/O设备，5) 外部事件

中断响应：

1) 响应中断的条件 2) 响应中断的时机

中断处理



时钟管理

时钟的重要性

时钟是计算机系统的脉搏。计算机的很多活动都是由**定时测量**来驱动的；
操作系统内核需要完成两种主要的定时测量，一是**保存当前的日期和时间**，二是**维持定时器**。

计算机系统时钟（两个时钟源）

实时时钟（RTC），也称 CMOS 时钟，是一块时钟芯片，靠电池供电。

OS 时钟，产生于 PC 主板上的定时/计数芯片，在开机时有效，由操作系统控制。

操作系统的时钟机制

时钟硬件，**保存当前的日期和时间**；

OS时钟管理硬件，即可编程间隔定时器 PIT：主要由 3 部分构成：**晶振、计数器和保持寄存器**。

时钟驱动程序，也称为时钟中断处理程序，**维持定时器**

其功能：

- ① 维护日期和时间
- ② 递减当前进程在一个时间片内的剩余执行时间，防止运行超时
- ③ 对CPU的使用情况记账
- ④ 递减报警计数器

系统调用

什么是系统调用

系统调用是一群**预先定义好的模块**，
提供**一条管道让应用程序能由此到核心程序**的服务。
系统调用是**系统程序与用户程序之间的接口**

系统调用与一般函数的区别

用户空间，用户进程所处的地址空间 系统空间，含有一切 系统核心代码 的地址空间	用户态执行，CPU执行用户空间的代码时，称该进程处于用户态执行 系统态执行，CPU执行 系统核心代码 时，称该进程处于 系统态执行
---	--

- 区别1:**系统调用运行在系统态，一般函数运行在用户态
- 区别2:**执行过程不同（系统调用执行时，当前进程被中断）
- 区别3:**系统调用进行中断处理，多了系统开销

系统调用的类型： **口诀：精简射虎童**

- 1) 进程控制类，
- 2) 文件操纵类，
- 3) 设备管理类，
- 4) 通信类，打开、关闭连接，交换信息
- 5) 信息维护类，返回系统当前日期、时间、版本号、用户数、空闲内存和磁盘大小等信息。

2.4 进程同步

进程同步的基本概念

进程并发执行，共享系统的资源，操作系统对共享过程进行控制和管理
 同步机制：保证在多任务共享系统资源的情况下，程序执行能得到正确的结果

多道程序环境下进程之间的关系	进程同步的任务
资源共享关系 相互合作关系	保证各进程以互斥的方式访问临界资源 保证相互合作的各进程协调执行

临界资源与临界区

- 必须以互斥方式访问的共享资源称为**临界资源**
- 进程中访问临界资源的那段**代码**称为**临界区**

同步机制应遵循的准则

```
{...
    进入区; //检查进程是否可以进入临界区并对临界区加锁🔒
    临界区;
    退出区; //释放临界区访问权（解锁）
...
}
```

进程同步的准则

- 1) 空闲让进，没有进程处于临界区，应允许一个请求进入临界区的进程进入；
- 2) 忙则等待，临界区已有进程，其他试图进入临界区的进程必须等待；
- 3) 有限等待，对于要访问临界资源的进程，应保证有限时间内进入临界区；
- 4) 让权等待，申请不到访问权，应释放处理机，以免浪费CPU资源。

信号量机制

用信号量的取值来表示资源的使用状况，以此为基础实现进程同步。

信号量，某种类型的变量，如整型、记录型

根据信号量类型的不同，信号量机制分：**整型信号量机制、记录型信号量机制、AND型信号量**

机制

整型信号量机制

整型信号量是**表示共享资源状态**且只能由特殊的**原子操作**改变的**整型量**

原理：定义一个**整型变量**，用该变量的值来**标记资源的使用情况**

- 整型信号量 >0 有资源可用；
- 整型信号量 <=0 资源忙，需等待

用于申请资源的是 wait 原子操作；用于释放资源的是 signal 原子操作。

用整型信号量实现进程互斥

为互斥访问的临界资源CS定义一个互斥信号量mutex，将初始值置为1
将临界资源CS放入wait(mutex) 和 signal(mutex)之间

记录型信号量机制

原理：定义一个**记录型变量**，用该变量的值来**标记资源的使用情况**

s.value >= 0时，s.value的值表示**资源数量**，

s.value < 0时，s.value的绝对值表示等待队列中**阻塞进程的数量**

记录型信号量的数据类型

```
Type semaphore = record
    Value: integer; //资源数量
    L: list of process; //阻塞队列
end
```

Var s semaphore;

```
wait(s)//请求资源
{
    s.value=s.value-1;
    if s.value<0 then block(s.L)
}
```

```
signal(s)//释放资源
{
    s.value=s.value+1;
    if s.value<=0 then
        wakeup(s.L)
}
```

AND型信号量机制

原理：将进程在整个运行过程中所需要的所有资源一次性地全部分配给进程，使用完毕一起释放。

AND型信号量机制的实现：

原子方法名：Swait(s1,s2,.....,sn)、Ssignal(s1,s2,.....,sn)

2.5 进程通信

进程通信

进程通信：进程间的信息交换

进程通信机制：

共享存储器系统：相互通信的进程共享某些数据结构或共享存储区；

消息传递系统：进程间通过操作系统提供的一组通信程序传递消息；

管道通信：进程间通过管道(链接读写进程的特殊文件)进行信息通信；

消息缓冲队列：利用消息缓冲区、发送原语和接收原语实现信息通信。

高级通信机制：**共享存储器系统、消息传递系统、管道通信。**

2.6 线程

线程的概念和分类

线程概念

程序: 具有独立功能的一组指令的集合

进程: 允许并发执行的程序在某个数据集合上的运行过程
线程

线程是进程中的一个实体，是被系统**独立调度和分配的基本单位**。

线程只拥有在运行中必需的资源，包括**程序计数器、一组寄存器和栈**，

但它可与同一进程的其他线程**共享进程所拥有的全部资源**。



线程的分类

- 1) 内核级线程，依赖于内核；
- 2) 用户级线程，不依赖内核。

线程的3种基本状态

线程的3种状态：就绪态、运行态、阻塞态

线程控制块

每一个线程都由一个数据结构表示，包括它的基本状态等，这个数据结构就是线程控制块TCB。
TCB记录了操作系统需要的、用于描述线程情况及控制线程运行所需的全部信息。

线程与进程的关系

- 1) 资源和调度。线程是程序执行的基本单位，进程是拥有资源的基本单位。
- 2) 地址空间资源。不同进程的地址空间是相互独立的，而同一进程中的各线程共享同一地址空间。
- 3) 通信关系。进程之间的通信必须使用操作系统提供的进程间通信机制，而同一进程中的各线程间可以直接通信。
- 4) 并发性。多进程之间可以并发执行，多线程之间也可以并发执行。
- 5) 系统开销。线程切换的开销比进程切换的开销小。

线程控制

四项基本操作功能：

- 1) 线程创建。
- 2) 线程的终止。
- 3) 线程的调度与切换。
- 4) 线程的阻塞与唤醒。

1) 用户线程的创建

2) 内核线程的创建

阻塞

唤醒

创建

调度

切换

终止

1) 请求系统服务

2) 启动某种操作

3) 新数据尚未到达

1) 正常结束

2) 异常结束

3) 外界干扰

1) 用户线程的调度与切换

2) 内核线程的调度与切换